

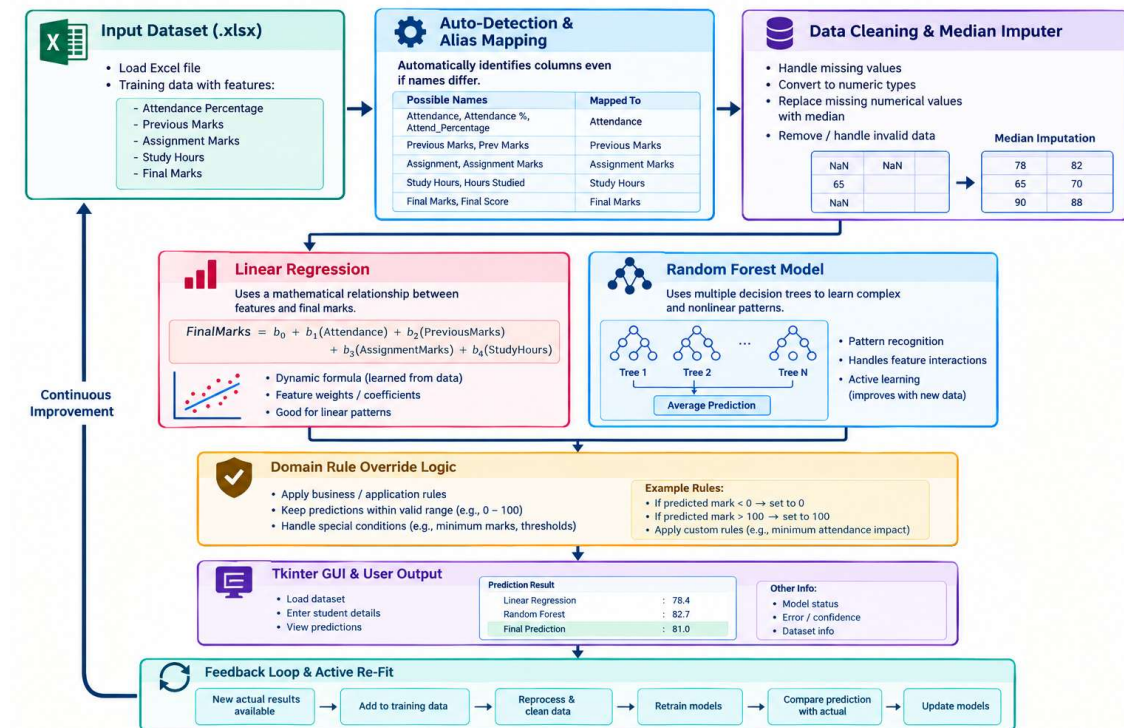
STUDYSENSE - Project Technical Documentation

1. Project Overview & Objective

STUDYSENSE is a machine learning-backed desktop GUI application built to predict students' final academic performance based on input metrics: Attendance, Previous Marks, Assignment Marks, and Study Hours. The primary goal is to provide transparent, dynamic predictive analytics that automatically adapt when user dataset structures or values change. Unlike static models, STUDYSENSE emphasizes user autonomy by allowing the underlying data engine to evolve alongside the institution's historical records.

2. Technical Architecture & Approach

The system follows a sequential pipeline designed for robustness and real-time responsiveness:



- 1.Data Ingestion:** The application loads raw Excel or CSV files.
- 2.Auto-Detection:** A fuzzy-matching algorithm scans headers to map user-defined names to internal variables.
- 3.Cleaning & Imputation:** Missing values are handled using median strategies to preserve dataset integrity.
- 4.Dual-Model Training:** The system concurrently fits a Linear Regression model for interpretability and a Random Forest Regressor for capturing high-dimensional patterns.
- 5.Domain Logic Layer:** Predictions are passed through a rule-based override system to ensure "common sense" academic outcomes.
- 6.GUI Presentation:** A Tkinter-based interface displays the mathematical formula and the calculated prediction.
- 7.Active Feedback Loop:** User-verified results are appended to the source file, triggering an immediate re-calculation of the predictive weights.

3. Detailed Methodology & Rationale

A. Column Auto-Detection & Schema Matching

- **Approach:** Standardized keyword dictionaries map non-standard user column names (e.g., `prev_marks`, `hours_studied`, `att_pct`) directly to internal canonical keys using substring and exact matching (`auto_detect_columns`).
- **Why this approach?:** User-provided Excel sheets rarely share identical column names. Hardcoding exact column names leads to frequent runtime errors. Alias mapping ensures schema resilience across different dataset source files, allowing for a "plug-and-play" user experience.

B. Missing Data Handling (Imputation)

- **Approach:** Utilized `sklearn.impute.SimpleImputer` configured with `strategy='median'`.
- **Why this approach?:** Dropping missing rows reduces dataset size significantly, which is detrimental on smaller academic files. Median imputation is robust against extreme outliers—such as a student with 0% attendance—compared to mean imputation, maintaining reliable central tendencies across academic metrics.

C. Model Selection Strategy (Dual-Model Pipeline)

Model Component	Primary Role	Technical Specification
Linear Regression	Interpretability Engine	Fits numerical features to compute variable coefficients and baseline intercepts.
Random Forest	Non-Linear Pattern Engine	Trained with 100 decision trees (<code>n_estimators=100</code>) as the underlying predictive engine.

- **Rationale (Linear Regression):** Deep learning and complex ensemble techniques operate as black boxes. Providing an explicit mathematical equation (e.g., $\text{Final Marks} = \text{Intercept} + \sum(\text{Coef}_i * \text{Feature}_i)$) builds user trust and makes the model's decision criteria completely transparent.
- **Rationale (Random Forest):** Random Forests excel at capturing complex feature interactions—such as the synergy between high assignment marks and moderate study hours—without requiring extensive hyperparameter tuning or feature scaling.

D. Exact-Match Optimization & Domain Rule Suppression

- **Approach:**
 - a. Checks for exact historical feature matches first. If an identical profile is found in the training data, the system returns the true empirical average.
 - b. Implements domain-specific rule logic: if $\text{Previous Marks} \geq 70\%$ and $\text{Attendance} \leq 60\%$, the system suppresses extreme attendance penalization in favor of past academic performance.
- **Why this approach?:** Pure mathematical models often over-penalize high-performing students who have low attendance. Applying a hybrid rule layer aligns raw algorithmic outputs with real-world academic patterns where prior aptitude often offsets low presence.

E. Active Learning & Dynamic Dataset Updates

- **Approach:** When a user verifies a predicted mark, the verified tuple is appended to the active dataset Excel file. This action triggers `analyze_dataset_and_generate_formula()` to re-fit the models instantly.
- **Why this approach?:** This creates a self-improving system. By continuously incorporating new data into the dataset, the formula coefficients and feature impacts update dynamically over time, reflecting changes in curriculum difficulty or grading standards.

4. Setup & Running Instructions

1. Install dependencies

Ensure you have Python 3.8+ installed. Execute the following command in your terminal:

```
pip install pandas numpy scikit-learn openpyxl
```

2. Run script

Launch the application by running the main entry point: `python studysense.py`